

Tome 1 — Laravel de A à Z, avec notre projet

Pour les 4 membres. Ce cours part de zéro et s'appuie uniquement sur **notre** repo : chaque exemple existe (ou a existé) dans `webcup-2026`. Chaque chapitre suit le même plan : **à quoi ça sert, comment on l'écrit chez nous, les erreurs fréquentes**. Les exercices sont au chapitre 26 : faites-les avant samedi, c'est là que se gagne la vitesse.

Ordre de lecture conseillé : tout le monde lit les chapitres 1 à 12 et 25. Ensuite, selon le rôle : Judicaël 13, 14, 18, 22 ; Tsoa 11, 15, 16, 21 ; Njaraniaina 13 et 22 ; Randy 23 et 24.

#	Chapitre	#	Chapitre
1	Laravel pour un développeur Spring / Angular	14	Filament : l'espace admin
2	Le trajet d'une requête	15	Fichiers et images
3	Les dossiers à connaître	16	Carte et géolocalisation
4	Routes	17	Notifications et e-mails
5	Eloquent : modèles et requêtes	18	Services externes : API de l'orga, IA
6	Relations	19	Cache, tâches planifiées, file d'attente
7	Migrations	20	Statistiques et export CSV
8	Validation	21	Données de démo : factories et seeders
9	Blade	22	Tests avec Pest
10	Livewire 4	23	Déboguer
11	Flux, Tailwind et identité visuelle	24	Déployer
12	Authentification, rôles et Politiques	25	Le générateur <code>make:feature</code>
13	Sécurité : les règles et les attaques	26	Exercices

1. Laravel pour un développeur Spring / Angular

Vous connaissez déjà les concepts ; seuls les noms changent.

Spring Boot / Angular	Laravel (chez nous)	Où
@Entity + JPA	Modèle Eloquent	app/Models/
JpaRepository	Le modèle lui-même : Signalement::where(...)->get()	—
Flyway / Liquibase	Migrations	database/migrations/
@Controller + Thymeleaf	Composant Livewire (PHP + vue dans un fichier)	resources/views/pages/
Composant Angular	Composant Livewire (état + template + événements)	idem
@PreAuthorize	Policy + \$this->authorize()	app/Policies/
Bean Validation	rules()	dans chaque composant
application.properties	.env + config/	racine
@Service	Classe dans app/Services/ , injectée ou via app(...)	app/Services/
Données de test	Factory + Seeder	database/factories/ , database/seaders/
JUnit	Pest	tests/Feature/
@Scheduled	Scheduler	routes/console.php
Back-office maison	Filament (généré)	app/Filament/ → /admin
mvn / ng CLI	php artisan	—

Différence majeure : pas d'API REST + front séparé. Livewire rend le HTML côté serveur et envoi de petites requêtes AJAX à chaque clic. **Conséquence de sécurité** : chaque méthode publique d'un composant est appelable depuis le navigateur, avec n'importe quel argument.

2. Le trajet d'une requête

Quand un utilisateur ouvre /signalements :

1. **Route** (routes/features.php) : l'URL pointe vers le composant pages::signalements.index . Le middleware auth exige une connexion, sinon redirection vers /login .
2. **Composant Livewire** (resources/views/pages/signalements/index.blade.php) : mount() s'exécute et vérifie les droits avec \$this->authorize('viewAny', Signalement::class) .
3. **Policy** (app/Policies/SignalementPolicy.php) : répond oui ou non. Non → **403**.
4. **Modèle** : la propriété calculée items() lit les données (Signalement::query()->with('user')->latest()->paginate(10)).
5. **Vue** (bas du même fichier) : HTML rendu avec les composants **Flux** (<flux:table> , <flux:button> ...) dans le layout layouts/app (menu de gauche).
6. **Ensuite**, chaque saisie (wire:model) ou clic (wire:click) renvoie une petite requête AJAX : Livewire exécute la méthode PHP et met à jour seulement ce qui a changé.

3. Les dossiers à connaître

app/Models/	← entités (une classe = une table)
app/Policies/	← qui a le droit de faire quoi
app/Filament/	← espace admin (/admin)
app/Services/	← appels externes (IA, API de l'orga)
app/Console/Commands/	← nos commandes (make:feature)
app/Providers/	← configuration au démarrage (sécurité, Fortify)
config/	← configuration (lit le .env)
database/migrations/	← structure des tables
database/factories/	← fausses données réalistes
database/seeders/	← remplissage de la base de démo
lang/fr.json, lang/fr/	← traductions françaises
resources/css/app.css	← thème (couleur d'accent)
resources/views/pages/	← toutes nos pages (composants Livewire)
resources/views/layouts/app/sidebar.blade.php	← menu de gauche
resources/views/welcome.blade.php	← page d'accueil
routes/web.php, routes/features.php	← routes
routes/console.php	← tâches planifiées
tests/Feature/	← tests Pest
.env	← secrets et réglages locaux (jamais commité)

On ne touche **pas** à `vendor/`, `node_modules/`, `public/build/`, `storage/` (générés).

4. Routes

Nos fonctionnalités sont dans `routes/features.php`, **dans le groupe `auth`** :

```
Route::middleware(['auth'])->group(function () {
    Route::livewire('signalements', 'pages::signalements.index')->name('signalements.index');
    Route::livewire('signalements/create', 'pages::signalements.form')->name('signalements.create');
    Route::livewire('signalements/{signalement}', 'pages::signalements.show')->name('signalements.show');
    Route::livewire('signalements/{signalement}/edit', 'pages::signalements.form')->name('signalements.edit');

    // make:feature:routes
});
```

- `'pages::signalements.index'` = le fichier `resources/views/pages/signalements/index.blade.php`.
- `{signalement}` : Laravel charge automatiquement le `Signalement` correspondant (*route model binding*) et renvoie **404** s'il n'existe pas.
- `->name(...)` : on génère toujours les URL par leur nom : `route('signalements.show', $signalement)`.

- **Page publique** (décision explicite) : la déclarer **en dehors** du groupe `auth`, en haut du fichier.
- Le commentaire `// make:feature:routes` est un **marqueur** du générateur : ne pas le supprimer.

Erreurs fréquentes : « Route [x] not defined » → nom mal écrit, vérifier avec `php artisan route:list --path=signalements`. Une route `{signalement}` déclarée **avant** `create` capture « create » comme un ID : garder l'ordre ci-dessus.

5. Eloquent : modèles et requêtes

```
#[Fillable(['titre', 'description', 'niveau', 'zone', 'photo', 'date_incident', 'latitude', 'longitude'])]
class Signalement extends Model
{
    use HasFactory;

    public const NIVEAU_OPTIONS = ['faible', 'moyen', 'critique'];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    protected function casts(): array
    {
        return [
            'date_incident' => 'date',
            'latitude' => 'decimal:7',
            'longitude' => 'decimal:7',
        ];
    }
}
```

- **#[Fillable]** : les seuls champs remplissables en masse (`new Signalement($data)`, `->update($data)`). `user_id`, `role`, `statut` réservés n'y sont **jamais** : c'est une protection, pas un oubli (chapitre 13).
- **casts()** : conversions automatiques (`date` → objet Carbon, `boolean`, `decimal:7`, `array` pour du JSON).
- **Constantes d'options** : une liste par enum, réutilisée dans la validation, les filtres, l'admin.
- Table déduite du nom : `Signalement` → `signalements`.

Requêtes courantes

```

Signalement::all(); // tout (à éviter sur de grosses tables)
Signalement::find(5); // par id, ou null
Signalement::findOrFail(5); // par id, ou 404
Signalement::where('niveau', 'critique')->latest()->get();
Signalement::whereBelongsTo(auth()->user())->count(); // ceux de l'utilisateur connecté
Signalement::query()
    ->when($niveau, fn ($q) => $q->where('niveau', $niveau)) // filtre optionnel
    ->when($search, fn ($q) => $q->where('titre', 'like', "%{$search}%"))
    ->latest()
    ->paginate(10);
$s->update(['titre' => 'Nouveau']);
$s->delete();

```

Scope (filtre réutilisable) :

```

// dans le modèle
public function scopeCritiques(Builder $query): void
{
    $query->where('niveau', 'critique');
}
// utilisation
Signalement::critiques()->latest()->get();

```

Erreurs fréquentes : « Add [titre] to fillable property » → le champ manque dans `#[Fillable]` (s'il est légitime). Un champ qui reste vide après `update()` → même cause. `Attempt to read property on null` → `find()` a renvoyé `null` : utiliser `findOrFail()`.

6. Relations

```

// Un utilisateur a plusieurs signalements – app/Models/User.php
public function signalements(): HasMany
{
    return $this->hasMany(Signalement::class);
}

// Un signalement concerne une zone – app/Models/Signalement.php
public function zone(): BelongsTo
{
    return $this->belongsTo(Zone::class);
}

// Plusieurs-à-plusieurs : un événement a des participants
public function participants(): BelongsToMany
{
    return $this->belongsToMany(User::class)->withTimestamps();
}

```

Migrations correspondantes :

```
$table->foreignId('zone_id')->nullable()->constrained()->nullOnDelete(); // BelongsTo
facultatif
$table->foreignId('user_id')->constrained()->cascadeOnDelete(); // BelongsTo
obligatoire

// table pivot pour BelongsToMany (nom au singulier, ordre alphabétique : evenement_user)
Schema::create('evenement_user', function (Blueprint $table) {
    $table->id();
    $table->foreignId('evenement_id')->constrained()->cascadeOnDelete();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->timestamps();
    $table->unique(['evenement_id', 'user_id']);
});
```

Utilisation :

```
$signalement->zone->nom; // relation chargée à la demande
$zone->signalements()->create($data); // crée en remplissant zone_id
$evenement->participants()->attach($user->id); // inscrire
$evenement->participants()->detach($user->id); // désinscrire
$evenement->participants()->toggle($user->id); // basculer
```

Le problème N+1 : afficher 10 signalements avec `$s->user->name` fait 11 requêtes. Toujours charger à l'avance :

```
Signalement::with(['user', 'zone'])->latest()->paginate(10);
Zone::withCount('signalements')->get(); // $zone->signalements_count
```

Relation polymorphe (commentaires sur plusieurs types d'entités) : `morphMany` / `morphTo`, colonnes `$table->morphs('commentable')`. À n'utiliser que si le sujet l'exige.

7. Migrations

```
Schema::create('signalements', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('titre');
    $table->text('description')->nullable();
    $table->string('niveau', 50)->index();
    $table->string('statut', 30)->default('en_attente')->index();
    $table->decimal('latitude', 10, 7)->nullable();
    $table->boolean('ouvert')->default(true);
    $table->date('date_incident');
    $table->timestamps();
});
```

Commande	Effet
<code>php artisan make:migration add_statut_to_signalements_table</code>	Nouvelle migration
<code>php artisan migrate</code>	Applique les nouvelles
<code>php artisan migrate:status</code>	Liste ce qui est passé
<code>php artisan migrate:fresh --seed</code>	LOCAL uniquement : tout effacer et recréer

Ajouter une colonne à une table existante :

```
public function up(): void
{
    Schema::table('signalements', function (Blueprint $table) {
        $table->string('statut', 30)->default('en_attente')->index();
    });
}

public function down(): void
{
    Schema::table('signalements', function (Blueprint $table) {
        $table->dropColumn('statut');
    });
}
```

Règles d'or

1. **Ne jamais modifier une migration déjà déployée** : en créer une nouvelle.
2. Une colonne ajoutée à une table **qui a déjà des lignes** doit être `nullable()` ou avoir un `default(...)`.
3. Tester sur MariaDB (en ligne) : SQLite est plus tolérant (types, `ALTER`, fonctions de date).

8. Validation

Dans un composant Livewire :

```
protected function rules(): array
{
    return [
        'titre' => ['required', 'string', 'max:255'],
        'description' => ['nullable', 'string', 'max:5000'],
        'niveau' => ['required', Rule::in(Signalement::NIVEAU_OPTIONS)],
        'capacite' => ['required', 'integer', 'min:0', 'max:100000'],
        'email_contact' => ['nullable', 'email', 'max:255'],
        'date_incident' => ['required', 'date', 'before_or_equal:today'],
        'zone_id' => ['nullable', 'exists:zones,id'],
        'photo' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
    ];
}

public function save(): void
{
    $validated = $this->validate(); // données propres, ou erreurs affichées sous les champs
    // ...
}
```

- Les messages sont **en français** (`lang/fr/validation.php`). Nom lisible d'un champ : section `attributes` du même fichier (`'date_incident' => 'date de l'incident'`).
- Règle sur mesure en une ligne : `'titre' => ['required', function ($attr, $value, $fail) { if (str_contains($value, 'http')) $fail('Pas de lien dans le titre.');` }] .
- Unicité en modification : `Rule::unique('zones', 'nom')->ignore($this->record?->id)` .

Toujours valider côté serveur, même si le champ HTML a `required` : le jury enverra des requêtes à la main.

9. Blade

```
{{ $signalement->titre }}                {{-- affiche en échappant le HTML (sûr) --}}
{{ $signalement->created_at->diffForHumans() }}    {{-- « il y a 3 heures » --}}
{{ $signalement->date_incident?->format('d/m/Y') }}

@if ($liste->isEmpty()) ... @else ... @endif
@foreach ($liste as $item) ... @endforeach
@forelse ($liste as $item) ... @empty Aucun élément. @endforelse

@auth ... @endauth            @guest ... @endguest
@can('update', $signalement) ... @endcan            {{-- selon la Policy --}}

<a href="{{ route('signalements.show', $signalement) }}" wire:navigate>Voir</a>
titre }}">
{{ __('Log out') }}                {{-- texte traduit via lang/fr.json --}}
```

Jamais `{!! $variable !!` avec une donnée saisie par un utilisateur (faille XSS).

Composant Blade réutilisable (`resources/views/components/stat.blade.php`) :

```
@props(['label', 'valeur', 'icone' => 'chart-bar'])
<flux:card class="flex items-center gap-4">
  <flux:icon :name="$icone" class="size-8 text-accent" />
  <div>
    <flux:text>{{ $label }}</flux:text>
    <flux:heading size="xl">{{ $valeur }}</flux:heading>
  </div>
</flux:card>
```

Utilisation : `<x-stat label="Signalements" :valeur="$total" icone="exclamation-triangle" />`.

10. Livewire 4

Un fichier = la logique en haut, la vue en bas. Exemple réel (formulaire généré, simplifié) :

```

<?php

use App\Models\Signalement;
use Flux\Flux;
use Illuminate\Validation\Rule;
use Livewire\Attributes\Locked;
use Livewire\Attributes>Title;
use Livewire\Component;
use Livewire\WithFileUploads;

new #[Title('Signalement')] class extends Component {
    use WithFileUploads;

    #[Locked] // le navigateur ne peut pas le changer
    public ?Signalement $record = null;

    public string $titre = ''; // lié à un champ par wire:model
    public string $niveau = 'faible';
    public $photo = null;

    public function mount(?Signalement $signalement = null): void // au chargement
    {
        if ($signalement?->exists) {
            $this->authorize('update', $signalement);
            $this->record = $signalement;
            $this->titre = $signalement->titre;
            $this->niveau = $signalement->niveau;
        } else {
            $this->authorize('create', Signalement::class);
        }
    }

    protected function rules(): array
    {
        return [
            'titre' => ['required', 'string', 'max:255'],
            'niveau' => ['required', Rule::in(Signalement::NIVEAU_OPTIONS)],
            'photo' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
        ];
    }

    public function save(): void // appelé par wire:submit="save"
    {
        $this->record
            ? $this->authorize('update', $this->record)
            : $this->authorize('create', Signalement::class);
    }
}

```

```

        $validated = $this->validate();

        if ($this->record) {
            $this->record->update($validated);
            $record = $this->record;
        } else {
            $record = new Signalement($validated);
            $record->user()->associate(auth()->user()); // jamais depuis le formulaire
            $record->save();
        }

        Flux::toast(variant: 'success', text: 'Signalement enregistré.');
```

`$this->redirectToRoute('signalements.show', $record, navigate: true);`

```

    }
}; ?>

<section class="w-full max-w-2xl space-y-6">
    <flux:heading size="xl" level="1">{{ $record ? 'Modifier' : 'Ajouter' }} un
    signalement</flux:heading>

    <form wire:submit="save" class="space-y-6">
        <flux:input wire:model="titre" label="Titre" required />
        <flux:select wire:model="niveau" label="Niveau">
            @foreach (\App\Models\Signalement::NIVEAU_OPTIONS as $option)
                <flux:select.option :value="$option">{{ ucfirst($option) }}
</flux:select.option>
            @endforeach
        </flux:select>
        <flux:button type="submit" variant="primary">Enregistrer</flux:button>
    </form>
</section>

```

Les attributs à connaître

Attribut	Rôle
<code>#[Title('...')]</code>	Titre de l'onglet
<code>#[Locked]</code>	Propriété non modifiable depuis le navigateur (IDs, enregistrements)
<code>#[Computed]</code>	Propriété calculée, mise en cache pendant la requête : <code>\$this->items</code>
<code>#[Url]</code>	Propriété synchronisée avec l'URL (<code>?search=...</code>) : filtres partageables
<code>#[On('evenement')]</code>	Méthode appelée quand un événement est émis

Liste avec recherche et pagination (extrait réel de `<index>`) :

```

use WithPagination;

#[Url(except: '')]
public string $search = '';

public function updatedSearch(): void { $this->resetPage(); } // revenir à la page 1

#[Computed]
public function items(): LengthAwarePaginator
{
    return Signalement::query()
        ->with('user')
        ->when($this->search !== '', fn ($q) => $q->where('titre', 'like', '%'.$this->search.'%'))
        ->latest()
        ->paginate(10);
}

public function delete(int $id): void
{
    $record = Signalement::findOrFail($id);
    $this->authorize('delete', $record); // ← sans cette ligne : faille
    $record->delete();
    Flux::toast(variant: 'success', text: 'Signalement supprimé.');
```

Les gestes côté vue

Besoin	Code
Lier un champ	<code>wire:model="titre"</code>
Mettre à jour à chaque frappe / après une pause	<code>wire:model.live / wire:model.live.debounce.300ms</code>
Appeler une méthode	<code>wire:click="toggleFavori({{ \$item->id }})"</code>
Envoyer un formulaire	<code><form wire:submit="save"></code>
Demander confirmation	<code>wire:confirm="Supprimer cet élément ?"</code>
Afficher pendant le chargement	<code><div wire:loading>Chargement...</div> (wire:loading.delay pour éviter le clignotement)</code>
Cibler une action	<code>wire:loading wire:target="save"</code>
Désactiver pendant l'envoi	<code>wire:loading.attr="disabled"</code>
Navigation sans rechargement	<code>wire:navigate</code> sur les liens
Rafraîchir toutes les 10 s	<code><div wire:poll.10s></code>
Clé de ligne dans une boucle	<code>wire:key="row-{{ \$item->id }}"</code>
Ne pas toucher à une zone (carte, graphique JS)	<code>wire:ignore</code>

Événements entre composants : `$this->dispatch('signalement-cree');` puis, dans un autre composant, `#[On('signalement-cree')] public function rafraichir(): void { unset($this->items); }`.

Erreurs fréquentes : « Cannot mutate locked property » → propriété `#[Locked]` modifiée : normal, c'est la protection. Une propriété qui ne se met pas à jour → `wire:model` sans `.live` ne s'envoie qu'à la prochaine action. « Property type not supported » → type non gérable par Livewire (utiliser `string`, `int`, `bool`, `array`, un modèle).

11. Flux, Tailwind et identité visuelle

Flux (version gratuite ; documentation : fluxui.dev, filtrer « free ») :

```

<flux:heading size="xl" level="1">Titre de page</flux:heading>
<flux:subheading>Sous-titre</flux:subheading>
<flux:text>Paragraphe secondaire</flux:text>

<flux:button variant="primary" icon="plus" :href="route('signalements.create')"
wire:navigate>Ajouter</flux:button>
<flux:button variant="ghost" size="sm" icon="pencil-square" />
<flux:button variant="danger" wire:click="delete({{ $item->id }})" wire:confirm="Supprimer
?">Supprimer</flux:button>

<flux:input wire:model="titre" label="Titre" description="Court et précis" required />
<flux:input wire:model.live.debounce.300ms="search" icon="magnifying-glass"
placeholder="Rechercher..." />
<flux:textarea wire:model="description" label="Description" rows="5" />
<flux:select wire:model.live="niveau" label="Niveau">
    <flux:select.option value="">Tous</flux:select.option>
    <flux:select.option value="critique">Critique</flux:select.option>
</flux:select>
<flux:checkbox wire:model="ouvert" label="Ouvert" />
<flux:switch wire:model.live="mine" label="Seulement les miens" />

<flux:badge color="red" size="sm">Critique</flux:badge>
<flux:card>Contenu encadré</flux:card>
<flux:callout icon="information-circle">Message d'information</flux:callout>
<flux:separator />
<flux:link :href="route('signalements.index')" wire:navigate>&larr; Retour</flux:link>

```

Tableau paginé :

```

<flux:table :paginate="$this->items">
    <flux:table.columns>
        <flux:table.column>Titre</flux:table.column>
        <flux:table.column>Niveau</flux:table.column>
    </flux:table.columns>
    <flux:table.rows>
        @foreach ($this->items as $item)
            <flux:table.row wire:key="row-{{ $item->id }}">
                <flux:table.cell>{{ $item->titre }}</flux:table.cell>
                <flux:table.cell><flux:badge size="sm">{{ $item->niveau }}</flux:badge>
            </flux:table.cell>
        </flux:table.row>
        @endforeach
    </flux:table.rows>
</flux:table>

```

Fenêtre modale :

```
<flux:modal.trigger name="confirmer"><flux:button>Ouvrir</flux:button></flux:modal.trigger>

<flux:modal name="confirmer" class="md:w-96">
  <flux:heading>Confirmer ?</flux:heading>
  <flux:button wire:click="valider" variant="primary">Oui</flux:button>
</flux:modal>
```

Message après une action (PHP) : `Flux::toast(variant: 'success', text: 'Enregistré.');`
 (variantes : `success`, `warning`, `danger`). Icônes : noms Heroicons (heroicons.com) : `map-pin`,
`bell`, `heart`, `chart-bar`, `exclamation-triangle`.

Les 4 états de chaque écran (le jury les remarque) :

```
@if ($this->items->isEmpty())
  <flux:card class="py-12 text-center">
    <flux:heading>Aucun signalement pour le moment</flux:heading>
    <flux:text class="mt-2">Soyez le premier à signaler un incident.</flux:text>
    <flux:button class="mt-4" variant="primary" :href="route('signalements.create')">
wire:navigate>Signaler</flux:button>
  </flux:card>
@else
  {{-- tableau --}}
@endif
<div wire:loading.delay class="text-sm text-zinc-500">Chargement...</div>
```

Tailwind — l'essentiel

Espacements	<code>p-4 px-6 py-2 m-4 mt-2 gap-4 space-y-6</code>
Mise en page	<code>flex flex-col items-center justify-between grid grid-cols-1 md:grid-cols-3</code>
Tailles	<code>w-full max-w-2xl h-40 size-10</code>
Texte	<code>text-sm text-lg font-medium font-semibold text-zinc-500</code>
Formes	<code>rounded-lg rounded-xl border shadow-sm</code>
Images	<code>object-cover aspect-video</code>
Sombre	<code>dark:bg-zinc-900 dark:text-white</code>
Responsive	<code>sm: (≥640px) md: (≥768px) lg: (≥1024px)</code>

Mobile d'abord : écrire pour le téléphone, puis ajouter `md: / lg: : grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3`.

Identité visuelle du sujet — deux endroits seulement :

```
/* resources/css/app.css : bloc @theme (mode clair) ET le bloc du mode sombre juste en dessous */
--color-accent: var(--color-orange-500);
--color-accent-content: var(--color-orange-600);
--color-accent-foreground: var(--color-white);
```

Changer les **deux** blocs (clair et sombre) ; en sombre, prendre une teinte plus claire (`orange-400`) pour garder le contraste.

```
// app/Providers/Filament/AdminPanelProvider.php
->colors(['primary' => Color::Orange])
->brandName(config('app.name'))
```

Plus : `APP_NAME` dans `.env`, le logo (`resources/views/components/app-logo.blade.php`), la page d'accueil (`welcome.blade.php`).

Ajouter une entrée au menu (le générateur le fait) : dans `sidebar.blade.php`, **au-dessus** du marqueur `{{-- make:feature:nav --}}` :

```
<flux:sidebar.item icon="map" :href="route('carte')" :current="request()->routeIs('carte')"
wire:navigate>
    Carte
</flux:sidebar.item>
```

Erreur fréquente : une classe Tailwind sans effet → `composer run dev` ne tourne pas ou le build est ancien : `npm run build`, puis Ctrl+F5.

12. Authentification, rôles et Politiques

Authentification : fournie par le starter kit (Fortify). Déjà en place : inscription, connexion, mot de passe oublié, **double authentification (2FA)**, confirmation du mot de passe, **limite de 5 essais par minute** (`FortifyServiceProvider`), mots de passe hachés. Le jury compte ces éléments comme des fonctionnalités de sécurité : **on les déclare**.

```
auth()->user();          // l'utilisateur connecté (ou null)
auth()->id();            // son id
auth()->check();         // connecté ?
```

Rôles : une colonne `role` sur `users` (`user` ou `admin`) et une méthode sur le modèle :

```
public function isAdmin(): bool
{
    return $this->role === 'admin';
}
```

`role` n'est **pas** dans `#[Fillable]` : un utilisateur ne peut pas s'inscrire « admin ». Pour promouvoir : `/admin/users` (admin), ou sur le serveur :

```
php84 artisan tinker --execute="App\Models\User::where('email', 'x@y.z')->update(['role' => 'admin']);"
```

Politiques : une classe par modèle, découverte automatiquement (`App\Policies\SignalementPolicy` pour `App\Models\Signalement`).


```

class SignalementPolicy
{
    public function viewAny(User $user): bool { return true; }
    public function view(User $user, Signalement $s): bool { return true; }
    public function create(User $user): bool { return true; }
    public function update(User $user, Signalement $s): bool
    {
        return $user->isAdmin() || $s->user_id === $user->id;
    }
    public function delete(User $user, Signalement $s): bool
    {
        return $user->isAdmin() || $s->user_id === $user->id;
    }
    public function moderer(User $user): bool           // action métier sur mesure
    {
        return $user->isAdmin();
    }
}

```

Utilisation :

```

$this->authorize('update', $signalement);           // composant Livewire → 403 si refusé
$this->authorize('create', Signalement::class);     // action sans instance
$user->can('moderer', Signalement::class);         // test booléen

```

```
@can('update', $signalement) <flux:button>Modifier</flux:button> @endcan
```

Règle réservée aux admins sans modèle (page de statistiques, par exemple) : dans `AppServiceProvider::boot()`, `Gate::define('admin', fn (User $user) => $user->isAdmin());`, puis sur la route `->middleware('can:admin')` et dans Blade `@can('admin')`.

Données privées : une donnée visible seulement par son propriétaire → `view()` renvoie `$user->isAdmin() || $s->user_id === $user->id`, **et** la liste filtre : `->when(! auth()->user()->isAdmin(), fn ($q) => $q->whereBelongsTo(auth()->user()))`.

13. Sécurité : les règles et les attaques

Ce que le jury teste, et où c'est géré chez nous

Famille testée	Protection	Où
Authentification	Fortify : hash, 2FA, confirmation	starter kit
Brute force	5 essais/minute sur la connexion (réponse 429)	FortifyServiceProvider , SecurityTest
Contrôle d'accès	Policies + <code>authorize</code> dans chaque action	<code>app/Policies/</code> , pages
Rôles	<code>role hors #[Fillable]</code> , /admin réservé	User , Filament
Validation	<code>rules()</code> sur chaque formulaire	pages <code><form</code>
Endpoints	Routes métier dans le groupe <code>auth</code>	<code>routes/features.php</code>
Exposition de données	<code>APP_DEBUG=false</code> , <code>#[Hidden]</code> sur les secrets de User	<code>.env</code> , User
En-têtes HTTP	X-Frame-Options, nosniff, Referrer-Policy, HSTS, HTTPS forcé	SecurityHeaders , AppServiceProvider
CSRF	Automatique (formulaires Blade et Livewire)	Laravel

Les 5 attaques qu'un juré fera en 10 minutes

1. **Changer l'ID dans l'URL** (`/signalements/12/edit` avec le compte d'un autre) → **403**.
2. **Appeler une action Livewire à la main** (`delete(12)` depuis la console du navigateur) → **403**.
3. **Ajouter un champ** à la requête (`role=admin` , `user_id=1` , `statut=valide`) → **ignoré**.
4. **Ouvrir** `/admin` avec un compte normal → **403**.
5. **Injecter du HTML/JS** (`<script>alert(1)</script>`) → affiché comme du texte.

Les parades, dans le code

```
// 1 et 2 : vérifier les droits dans CHAQUE méthode publique
public function delete(int $id): void
{
    $signalement = Signalement::findOrFail($id);
    $this->authorize('delete', $signalement);
    $signalement->delete();
}

// 1 : verrouiller l'enregistrement du composant
#[Locked]
public ?Signalement $record = null;

// 3 : champs sensibles assignés dans le code, jamais depuis le formulaire
$signalement = new Signalement($validated);
$signalement->user()->associate(auth()->user());
$signalement->statut = 'en_attente';
$signalement->save();
```

Limiter une action sensible (envoi, signalement, appel IA) :

```

use Illuminate\Support\Facades\RateLimiter;

$cle = 'signalement:'.auth()->id();
if (RateLimiter::tooManyAttempts($cle, 10)) {
    $this->addError('titre', 'Trop de signalements. Réessayez dans une minute.');
```

```
    return;
```

```
}
```

```
RateLimiter::hit($cle, 60);
```

Injection SQL : Eloquent protège automatiquement. Seul danger : `whereRaw` / `DB::raw` avec une valeur utilisateur concaténée. Toujours des paramètres : `->whereRaw('LOWER(titre) LIKE ?', ['%'.$s.'%'])`.

Autres règles : uploads limités (chapitre 15) ; messages d'erreur génériques ; aucune donnée personnelle d'autrui affichée sans règle ; clés d'API uniquement côté serveur ; texte inséré en JavaScript via `textContent`, jamais `innerHTML`.

Annonces « sécurité » probables et réponses prêtes

Annonce	Réponse
« Authentification sécurisée », « Limiter les tentatives de connexion »	Déjà là → déclarer et montrer
« Rôles utilisateur / administrateur »	Déjà là → déclarer, montrer <code>/admin/users</code>
« Protéger une zone sensible »	Policy ou <code>can:admin</code> + test 403 (15 min)
« Validation renforcée »	Compléter <code>rules()</code> (tailles, formats, <code>Rule::in</code>)
« Journal des actions »	Modèle <code>ActionLog</code> + ressource Filament en lecture seule (30 min)
« Limiter le spam »	<code>RateLimiter</code> (15 min)
« Masquer les données sensibles »	Policy <code>view</code> restrictive + masquage dans les vues

Journal des actions (exemple complet) :

```
// migration
$table->id();
$table->foreignId('user_id')->nullable()->constrained()->onDelete('nullOnDelete');
$table->string('action', 50); // created, updated, deleted, login...
$table->string('subject_type')->nullable();
$table->unsignedBigInteger('subject_id')->nullable();
$table->string('ip', 45)->nullable();
$table->timestamps();

// app/Models/ActionLog.php
#[Fillable(['user_id', 'action', 'subject_type', 'subject_id', 'ip'])]
class ActionLog extends Model
{
    public static function record(string $action, ?Model $subject = null): void
    {
        static::create([
            'user_id' => auth()->id(),
            'action' => $action,
            'subject_type' => $subject ? class_basename($subject) : null,
            'subject_id' => $subject?->getKey(),
            'ip' => request()->ip(),
        ]);
    }
}
```

Appel : `ActionLog::record('deleted', $signalement);`. Ici `user_id` est remplissable car il ne vient **jamais** d'un formulaire.

14. Filament : l'espace admin

Tout ce qui est dans `/admin` est réservé aux admins (`User::canAccessPanel()`), et les Politiques s'y appliquent aussi.

```
php artisan make:filament-resource Signalement --generate
```

Crée `app/Filament/Resources/Signalements/` : la ressource, `Schemas/SignalementForm.php` (formulaire), `Tables/SignalementsTable.php` (liste), `Pages/` (liste, création, modification). **Le code généré doit toujours être corrigé :**

Formulaire

```

use Filament\Forms\Components\{DatePicker, FileUpload, Select, Textarea, TextInput, Toggle};

return $schema->components([
    TextInput::make('titre')->required()->maxLength(255),
    Textarea::make('description')->columnSpanFull(),
    Select::make('niveau')
        ->options(array_combine(Signalement::NIVEAU_OPTIONS, array_map('ucfirst',
Signalement::NIVEAU_OPTIONS)))
        ->required(),
    FileUpload::make('photo')->image()->disk('public')->directory('signalements')->maxSize(2048),
    DatePicker::make('date_incident')->required(),
    Toggle::make('ouvert'),
]);

```

⚠ Le générateur Filament ajoute `Select::make('user_id')`. Comme `user_id` est **hors** `#[Fillable]`, la création échouerait. Deux choix : **supprimer ce champ** (l'admin modère, il ne crée pas au nom des autres), ou assigner explicitement dans `Pages/CreateSignalement.php` :

```

protected function handleRecordCreation(array $data): Model
{
    $record = new Signalement($data);
    $record->user_id = $data['user_id'];
    $record->save();

    return $record;
}

```

Tableau

```

use Filament\Actions\Action, EditAction;
use Filament\Tables\Columns\IconColumn, ImageColumn, TextColumn;
use Filament\Tables\Filters\SelectFilter;

return $table
    ->columns([
        ImageColumn::make('photo')->disk('public')->circular(),
        TextColumn::make('titre')->searchable()->limit(40),
        TextColumn::make('user.name')->label('Auteur')->searchable(),
        TextColumn::make('niveau')->badge()->color(fn (string $state) => match ($state) {
            'critique' => 'danger', 'moyen' => 'warning', default => 'gray',
        }),
        IconColumn::make('ouvert')->boolean(),
        TextColumn::make('created_at')->label('Créé le')->dateTime('d/m/Y H:i')->sortable(),
    ])
    ->defaultSort('created_at', 'desc')
    ->filters([
        SelectFilter::make('niveau')->options(array_combine(Signalement::NIVEAU_OPTIONS,
Signalement::NIVEAU_OPTIONS)),
    ])
    ->recordActions([
        Action::make('valider')
            ->label('Valider')
            ->icon('heroicon-o-check')
            ->color('success')
            ->requiresConfirmation()
            ->visible(fn (Signalement $record) => $record->statut === 'en_attente')
            ->action(function (Signalement $record) {
                $record->statut = 'valide'; // hors Fillable : assignation explicite
                $record->save();
            }),
        EditAction::make(),
    ]);

```

Libellés en français dans la ressource : `protected static ?string $modelLabel = 'signalement';`, `protected static ?string $pluralModelLabel = 'signalements';`, icône `protected static string|BackedEnum|null $navigationIcon = Heroicon::OutlinedExclamationTriangle;`.

Ressource en lecture seule (journal) : `public static function canCreate(): bool { return false; }` et pas d' `EditAction`.

Statistiques sur le tableau de bord admin

```
php artisan make:filament-widget StatsOverview --stats-overview
```

```

use Filament\Widgets\StatsOverviewWidget\Stat;

protected function getStats(): array
{
    return [
        Stat::make('Signalements', Signalement::count()),
        Stat::make('Critiques', Signalement::where('niveau', 'critique')->count())-
        >color('danger'),
        Stat::make('Utilisateurs', User::count()),
    ];
}

```

Graphique : `php artisan make:filament-widget SignalementsParJour --chart` puis `getType()` → `'line'` et `getData()` → `['datasets' => [['label' => 'Signalements', 'data' => $valeurs]], 'labels' => $jours]`.

Rappels : ne jamais afficher un mot de passe, un secret 2FA ou un jeton ; un champ hors `# [Fillable]` modifié depuis Filament doit être assigné explicitement (voir `Pages/EditUser.php`), sinon il est ignoré sans erreur.

15. Fichiers et images

```

use Livewire\WithFileUploads;

use WithFileUploads;
public $photo = null;

// règle
'photo' => ['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],

// enregistrement (nom aléatoire, disque public)
if ($this->photo) {
    if ($this->record->photo) {
        Storage::disk('public')->delete($this->record->photo); // supprimer l'ancienne
    }
    $validated['photo'] = $this->photo->store('signalements', 'public');
} else {
    unset($validated['photo']); // ne pas écraser la photo existante par null
}

```

```

<flux:input type="file" wire:model="photo" label="Photo"
accept="image/jpeg,image/png,image/webp" />
<div wire:loading wire:target="photo"><flux:text>Envoi en cours...</flux:text></div>
@if ($photo)
    
@endif

{{-- affichage --}}
titre }}" class="aspect-video w-full
rounded-xl object-cover">

```

Plusieurs fichiers : `public array $photos = [];`, règle `'photos.*' => ['image', 'max:2048']`,
`<flux:input type="file" wire:model="photos" multiple />`.

`php artisan storage:link` (une fois, fait par `deploy.sh` en ligne) : sans lui, les images sont invisibles. Toujours un `alt`, toujours `object-cover`.

16. Carte et géolocalisation

Fonctionnalité fréquente (points de regroupement, incidents, services proches). **Leaflet**, sans installation :

```

#[Computed]
public function points(): array
{
    return Signalement::whereNotNull('latitude')
        ->get(['id', 'titre', 'latitude', 'longitude'])
        ->map(fn ($s) => ['lat' => (float) $s->latitude, 'lng' => (float) $s->longitude,
'titre' => $s->titre])
        ->all();
}

```



```
@assets
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
@endassets

<div wire:ignore
  x-data="{ points: @js($this->points) }"
  x-init="
    const map = L.map($el).setView([-18.91, 47.52], 13);
    L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', { attribution: '©
OpenStreetMap' }).addTo(map);
    points.forEach(p => {
      const texte = document.createElement('span');
      texte.textContent = p.titre;           // texte brut : aucune injection possible
      L.marker([p.lat, p.lng]).addTo(map).bindPopup(texte);
    });
  "
  class="h-96 w-full rounded-xl"></div>
```

- `wire:ignore` : Livewire ne redessine pas la carte.
- Centre par défaut : Antananarivo (-18.91, 47.52), à adapter.

« **Autour de moi** » : dans le `x-init`, `navigator.geolocation.getCurrentPosition(pos => $wire.set('lat', pos.coords.latitude))` (idem `lng`). Autorisé par nos en-têtes (`geolocation=(self)`), nécessite HTTPS (le cas en ligne). Puis trier par distance (approximation suffisante à l'échelle d'une ville) :

```
Signalement::query()
->whereNotNull('latitude')
->orderByRaw('(POW(latitude - ?, 2) + POW(longitude - ?, 2))', [$this->lat, $this->lng])
->limit(5)->get();
```

Choisir un point sur la carte (formulaire) : `map.on('click', e => { $wire.set('latitude', e.latlng.lat); $wire.set('longitude', e.latlng.lng); })`.

17. Notifications et e-mails

```
php artisan make:notification SignalementCritique
php artisan make:notifications-table && php artisan migrate      # notifications dans
l'application
```

```

class SignalementCritique extends Notification
{
    public function __construct(public Signalement $signalement) {}

    public function via(object $notifiable): array
    {
        return ['database', 'mail'];
    }

    public function toArray(object $notifiable): array
    {
        return ['titre' => $this->signalement->titre, 'url' => route('signalements.show',
$this->signalement)];
    }

    public function toMail(object $notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject('Nouveau signalement critique')
            ->line($this->signalement->titre)
            ->action('Voir', route('signalements.show', $this->signalement));
    }
}

```

Envoi : `Notification::send(User::where('role', 'admin')->get(), new SignalementCritique($s));`
ou `$user->notify(...)`.

Affichage (cloche) : `auth()->user()->unreadNotifications()->count()`, liste `auth()->user()->notifications()->latest()->take(10)->get()`, marquer lu : `$notification->markAsRead()`.

E-mails : en local `MAIL_MAILER=log` (les e-mails s'écrivent dans `storage/logs/laravel.log`) ; sur le serveur `MAIL_MAILER=sendmail` (testé jeudi).

18. Services externes : API de l'orga, IA

Configuration : jamais d'URL ni de clé dans le code.

```

# .env
ORGA_API_URL=https://...
ORGA_API_TOKEN=...
OPENROUTER_API_KEY=...
OPENROUTER_MODEL=nom-du-modele:free

```

```

// config/services.php
'orga' => ['url' => env('ORGA_API_URL'), 'token' => env('ORGA_API_TOKEN')],
'openrouter' => ['key' => env('OPENROUTER_API_KEY'), 'model' => env('OPENROUTER_MODEL')],

```

Client de l'API de l'orga (cache, délai, nouvel essai, repli) :

```
// app/Services/OrgaApi.php
class OrgaApi
{
    /** @return array<int, array<string, mixed>> */
    public function alertes(): array
    {
        try {
            return Cache::remember('orga:alertes', now()->addMinutes(5), fn () =>
Http::baseUrl(config('services.orga.url'))
                ->withToken(config('services.orga.token'))
                ->acceptJson()
                ->timeout(10)
                ->retry(2, 500)
                ->get('/alertes')
                ->throw()
                ->json('data', []));
        } catch (\Throwable $e) {
            report($e);
            return []; // la page affiche « Données momentanément indisponibles »
        }
    }
}
```

Utilisation : `app(OrgaApi::class)->alertes()` . Importer en base : une commande `php artisan make:command SyncAlertes` avec `Alerte::updateOrCreate(['external_id' => $d['id']], [...])` , planifiée (chapitre 19).

IA (OpenRouter) — la clé ne quitte jamais le serveur :

```
// app/Services/Ai.php
class Ai
{
    public function ask(string $system, string $prompt): ?string
    {
        if (! config('services.openrouter.key')) {
            return null;
        }

        return Cache::remember('ai:'.md5($system.$prompt), now()->addHour(), function () use
($system, $prompt) {
            $response = Http::withToken(config('services.openrouter.key'))
                ->timeout(25)
                ->post('https://openrouter.ai/api/v1/chat/completions', [
                    'model' => config('services.openrouter.model'),
                    'messages' => [
                        ['role' => 'system', 'content' => $system],
                        ['role' => 'user', 'content' => $prompt],
                    ],
                ],
            );

            return $response->successful() ? $response->json('choices.0.message.content') :
null;
        });
    }
}
```

Dans le composant : **limiter par utilisateur** (chapitre 13, 5 demandes/minute), afficher « Service indisponible, réessayez » si `null`, et afficher la réponse avec `{{ }}` (jamais comme du HTML). Limites OpenRouter : 20 requêtes/minute, 50/jour (1000 après un premier crédit). Le cache évite de consommer le quota deux fois pour la même question.

19. Cache, tâches planifiées, file d'attente

Cache : `Cache::remember('stats:accueil', 60, fn () => [...])` (60 secondes). Vider : `php artisan cache:clear`.

Tâches planifiées : dans `routes/console.php`

```
use Illuminate\Support\Facades\Schedule;

Schedule::command('app:sync-alertes')->everyFiveMinutes();
Schedule::call(fn () => Signalement::where('statut', 'en_attente')->where('created_at', '<',
now()->subDays(7))->update(['statut' => 'expire']))->daily();
```

Sur le serveur, une tâche cron cPanel lance le planificateur chaque minute :

```
* * * * * cd ~/webcup-2026 && /opt/cpanel/ea-php84/root/usr/bin/php artisan schedule:run >> /dev/null 2>&1
```

En local : `php artisan schedule:work` (ou `schedule:run` pour un passage). Liste : `php artisan schedule:list`.

File d'attente : en production, `QUEUE_CONNECTION=sync` (tout s'exécute tout de suite, pas de worker à surveiller). Suffisant pour nos volumes.

20. Statistiques et export CSV

```
Signalement::count();
Signalement::where('niveau', 'critique')->count();
Signalement::selectRaw('niveau, count(*) as total')->groupBy('niveau')->pluck('total', 'niveau');
// ['faible' => 12, 'moyen' => 7, 'critique' => 3]
Signalement::where('created_at', '>=', now()->subDays(7))->count();
Zone::withCount('signalements')->orderByDesc('signalements_count')->take(5)->get();
```

Page de tableau de bord utilisateur : des `<x-stat>` (chapitre 9) dans une grille. Barres simples sans bibliothèque :

```
@foreach ($this->parNiveau as $niveau => $total)
    <div class="flex items-center gap-3">
        <span class="w-20 text-sm">{{ ucfirst($niveau) }}</span>
        <div class="h-3 rounded bg-accent" style="width: {{ $max ? round($total / $max * 100) : 0 }}%"></div>
        <span class="text-sm">{{ $total }}</span>
    </div>
@endforeach
```

Export CSV depuis un composant Livewire :

```

public function export()
{
    $this->authorize('viewAny', Signalement::class);

    return response()->streamDownload(function () {
        $out = fopen('php://output', 'w');
        fwrite($out, "\xEF\xBB\xBF"); // accents lisibles dans
Excel
        fputcsv($out, ['Titre', 'Niveau', 'Auteur', 'Date'], ';');
        Signalement::with('user')->latest()->chunk(200, function ($lignes) use ($out) {
            foreach ($lignes as $s) {
                fputcsv($out, [$s->titre, $s->niveau, $s->user?->name, $s->created_at-
>format('d/m/Y')], ';');
            }
        });
        fclose($out);
    }, 'signalements-' . now()->format('Ymd') . '.csv');
}

```

Bouton : `<flux:button wire:click="export" icon="arrow-down-tray">Exporter</flux:button>`.

21. Données de démo : factories et seeders

Une application vide paraît inachevée. Chaque entité a une factory **crédible** :

```

// database/factories/SignalementFactory.php
public function definition(): array
{
    return [
        'user_id' => User::factory(),
        'titre' => fake()->randomElement([
            'Inondation rue Andrianampoinimerina', 'Arbre tombé à Isoraka', 'Coupure
d'électricité à Ivandry',
        ]),
        'description' => fake('fr_FR')->paragraph(),
        'niveau' => fake()->randomElement(Signalement::NIVEAU_OPTIONS),
        'latitude' => fake()->latitude(-18.95, -18.85),
        'longitude' => fake()->longitude(47.48, 47.56),
        'created_at' => fake()->dateTimeBetween('-10 days', 'now'),
    ];
}

public function critique(): static // état nommé
{
    return $this->state(fn () => ['niveau' => 'critique']);
}

```

```
// database/seeder/DatabaseSeeder.php (sous le marqueur // make:feature:seeders)
Signalement::factory(20)->recycle($users)->create();
Signalement::factory(3)->critique()->recycle($users)->create();
```

`recycle($users)` réutilise les utilisateurs existants. En local : `php artisan migrate:fresh --seed`
→ comptes `admin@example.com` / `user@example.com` , mot de passe `password` .

En production, jamais `db:seed` complet (il ne crée d'ailleurs les comptes de test que hors production). Pour remplir l'application en ligne : un seeder dédié **sans aucun compte** :

```
php artisan make:seeder DemoContentSeeder
php84 artisan db:seed --class=DemoContentSeeder --force # sur le serveur
```

Ou saisir les données de démo **via l'application**, avec les comptes jury : c'est aussi un test.

22. Tests avec Pest

```
use App\Models\{Signalement, User};
use Livewire\Livewire;

test('un invité est redirigé vers la connexion', function () {
    $this->get(route('signalements.index'))->assertRedirect(route('login'));
});

test('un autre utilisateur ne peut pas modifier', function () {
    $signalement = Signalement::factory()->create();

    $this->actingAs(User::factory()->create())
        ->get(route('signalements.edit', $signalement))
        ->assertForbidden();
});

test('le formulaire refuse un niveau inventé', function () {
    Livewire::actingAs(User::factory()->create())
        ->test('pages::signalements.form')
        ->set('titre', 'Test')
        ->set('niveau', 'apocalypse')
        ->call('save')
        ->assertHasErrors(['niveau']);
});

test('on ne peut pas se donner un statut réservé', function () {
    Livewire::actingAs($user = User::factory()->create())
        ->test('pages::signalements.form')
        ->set('titre', 'Test')
        ->set('niveau', 'faible')
        ->call('save');

    expect(Signalement::where('user_id', $user->id)->first()->statut)->toBe('en_attente');
});

test('un admin peut valider', function () {
    $admin = User::factory()->admin()->create();
    // ...
});
```

Simuler l'extérieur :


```
Http::fake(['openrouter.ai/*' => Http::response(['choices' => [['message' => ['content' =>
'Résumé']]]]]]);
Notification::fake(); /* ... */ Notification::assertSentTo($admin,
SignalementCritique::class);
Storage::fake('public');
->set('photo', UploadedFile::fake()->image('photo.jpg'))
```

Lancer : `php artisan test` (tout), `php artisan test --filter=Signalement` (un fichier), `php artisan test --parallel` (plus rapide).

À savoir : les tests tournent en anglais (`APP_LOCALE=en` dans `phpunit.xml`) car ceux du starter kit vérifient des textes anglais. La CI lance aussi **Pint** et **PHPStan** : une erreur de type bloque la PR.

Le minimum par fonctionnalité : « le propriétaire peut », « un autre reçoit 403 », « une valeur invalide est refusée », et un test par règle métier.

23. Débuguer

Symptôme	Réflexe
Page blanche / erreur 500	<code>storage/logs/laravel.log</code> (la fin du fichier)
Changement de style invisible	<code>composer run dev</code> doit tourner, ou <code>npm run build</code> ; Ctrl+F5
« Route not defined »	<code>php artisan route:list --path=...</code>
Champ non enregistré	Dans <code>#[Fillable]</code> ? Dans <code>rules()</code> ?
403 inattendu	Relire la Policy et l'appel <code>authorize</code>
« Column not found » / « no such table »	<code>php artisan migrate</code>
Modification non prise en compte	<code>php artisan optimize:clear</code>
Livewire ne réagit pas	Console du navigateur (F12) ; <code>wire:model.live</code> ? <code>wire:key</code> dans les boucles ?
Une valeur mystère	<code>dd(\$variable);</code> (affiche et arrête) ou <code>logger(\$variable);</code> (dans le log)

Outils : `php artisan tinker` (console PHP sur l'application : `Signalement::latest()->first()`), `php artisan about` (état général). Avec Claude Code : coller **l'erreur exacte** et les 20 dernières lignes du log, jamais « ça ne marche pas ». Retirer tous les `dd()` avant de commiter.

24. Déployer

deploy.sh (lancé par Randy ou Judicaël dans le Terminal cPanel) :

```
bash ~/webcup-2026/deploy.sh
```

1. `git pull` de `main` ;
2. `composer install --no-dev` ;

3. `npm ci` + `npm run build` ;
4. **sauvegarde de la base** (`~/backups` , 10 dernières) ;
5. `migrate --force` ;
6. `storage:link` , `filament:assets` , `optimize` ;
7. **contrôle de santé** (`/up` doit répondre 200).

Journal : `~/deploy.log` . Le script s'arrête à la première erreur.

Le **.env du serveur** (jamais dans Git) doit contenir notamment :

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://virtualvisionsy.madagascar.webcup.hodi.cloud
APP_LOCALE=fr
APP_FAKER_LOCALE=fr_FR
DB_CONNECTION=mariadb          (ou mysql)
QUEUE_CONNECTION=sync
MAIL_MAILER=sendmail
OPENROUTER_API_KEY=...
```

Après une modification du `.env` : `php84 artisan optimize` .

Ce qu'on ne fait jamais en production : `migrate:fresh` , `db:seed` , `key:generate` , `APP_DEBUG=true` , `git reset --hard` , modifier du code directement sur le serveur, supprimer le bloc `AddHandler` de `public/.htaccess` .

Revenir en arrière : en local `git revert <sha>` , `push` , `deploy.sh` . En dernier recours, restaurer la base : `gunzip -c <sauvegarde>.sql.gz | mysql -u <utilisateur> -p <base>` .

25. Le générateur `make:feature`

```
php artisan make:feature PointRegroupement \
--
fields="nom:string,adresse:string,capacite:integer,ouvert:boolean,niveau:enum(faible/moyen/elev
e),photo:image?,latitude:decimal?,longitude:decimal?" \
--label="Point de regroupement" --plural="Points de regroupement" --icon=map-pin
php artisan migrate
```

(Sous PowerShell, tout sur **une seule ligne**, sans les `\`.)

Type	Colonne	Champ du formulaire	Validation
string	string	flux:input	string , max:255
text	text	flux:textarea	string , max:5000
integer	integer	flux:input type=number	integer
decimal	decimal(10,7) si le nom contient latitude/longitude, sinon decimal(12,2)	flux:input type=number step=any	numeric
boolean	boolean	flux:checkbox	boolean
date / datetime	date / dateTime	flux:input type=date / datetime-local	date
enum(a/b/c)	string indexé + constante X_OPTIONS	flux:select + filtre dans la liste	Rule::in
image	chemin	flux:input type=file + aperçu	image , mimes , max:2048

Suffixe ? = facultatif (colonne nullable , règle nullable).

Ce qu'il génère : migration (avec `user_id`), modèle (#[Fillable] sans `user_id`, casts, constantes), factory (`fr_FR`, coordonnées à Antananarivo), policy (lecture/création pour tous les connectés, modification/suppression par le propriétaire ou un admin), 3 pages (liste avec recherche, filtres, « mes éléments », pagination, suppression ; formulaire avec validation et upload ; détail), routes, entrée de menu, ligne de seeder, 6 tests.

Marqueurs (ne jamais les supprimer) : `// make:feature:routes (routes), {{-- make:feature:nav --}}` (menu), `// make:feature:seeders (seeder)`.

Après génération, toujours : relire la migration, adapter la policy au sujet, rendre la factory crédible, ajuster textes et colonnes, puis `make:filament-resource ... --generate` si l'admin doit gérer l'entité (et corriger le formulaire, chapitre 14). `--force` régénère en écrasant (perte des adaptations).

26. Exercices

À faire sur une branche `exo/<prénom>` (**jamais mergée**), en local, dans l'ordre. Chaque exercice indique le résultat attendu.

- Explorer** (15 min) — `php artisan route:list --path=signalements`, ouvrir chaque route, lire `<index.blade.php` de haut en bas. *Attendu* : savoir dire ce que fait chaque méthode.
- Ajouter une colonne** (20 min) — migration `statut` (`en_attente` par défaut) sur `signalements`, badge dans la liste. *Attendu* : les anciens signalements affichent « en_attente ».
- Générer une entité** (15 min) — `make:feature Zone --fields="nom:string,description:text?"`. *Attendu* : menu « Zones », CRUD complet, 6 tests verts.
- Relier deux entités** (30 min) — `zone_id` sur `signalements`, `flux:select` des zones dans le formulaire, filtre par zone dans la liste, `with('zone')`. *Attendu* : pas de N+1 (une seule

requête pour les zones).

5. **Statut réservé à l'admin** (40 min) — action Filament « Valider », test « un utilisateur ne peut pas se valider lui-même ». *Attendu* : le test passe, l'action n'apparaît que sur les signalements en attente.
6. **Limiter une action** (20 min) — 3 signalements maximum par minute et par utilisateur, test associé. *Attendu* : le 4^e affiche un message en français.
7. **Carte** (30 min) — page « Carte » avec les signalements géolocalisés (chapitre 16), entrée de menu. *Attendu* : un titre contenant `<b>test</b>` s'affiche tel quel dans la bulle.
8. **Export CSV** (20 min) — bouton sur la liste. *Attendu* : fichier lisible dans Excel, accents corrects.
9. **Le cycle complet** (20 min) — sur une vraie branche `docs/<prénom>`, une petite modification, PR avec `Refs #`, CI verte, merge par Randy, déploiement, vérification en ligne.

Bloqué plus de 20 minutes ? Demander à Judicaël ou à Claude Code, avec l'erreur exacte.